Atelier 5 : examen de fin de module M202 « Approche agile », exercices Git (réponses)

Stagiaire : Ali EL Habib · OFPPT Tanger · Développement Digital
Dépôt : `depot_local` · branche principale : `master` (comme dans l'énoncé)

Partie 2 : gestion de version avec Git (7 points)

> Chaque réponse donne toutes les commandes, y compris la création des dossiers et les changements de dossier. Les commandes ont réellement été exécutées pour construire ce dépôt.

1. Créer `depot_local`, y entrer et l'initialiser comme dépôt Git (1 pt)

```
mkdir depot_local
cd depot_local
git init
```

2. Lier `depot_local` à un dépôt distant (1 pt)

```
git remote add origin https://github.com/alilhbib20007-sys/depot_local.git
git remote -v
```

3. Stager les trois fichiers vides et commiter (1 pt)

Fichiers créés avec VS Code : `produit.html`, `style.css`, `script.js` (on peut aussi faire `touch produit.html style.css script.js`).

```
git add produit.html style.css script.js      # ou : git add .
git commit -m "Version initiale : produit.html, style.css, script.js"
```

4. Créer la branche « Gestion produits » et basculer dessus (1 pt)

Un nom de branche Git ne peut pas contenir d'espace, donc on écrit `gestion-produits` :

```
git checkout -b gestion-produits
# équivalent : git branch gestion-produits  puis  git checkout gestion-produits
# ou encore  : git switch -c gestion-produits
```

5. Commiter les modifications, revenir sur `master` et fusionner (1 pt)

```
git add .
git commit -m "Gestion produits : contenu de produit.html, style.css et script.js"
git checkout master
git merge gestion-produits
```

6. Log sur une ligne, informations de `origin`, puis push (1 pt)

```
git log --oneline
git remote show origin        # informations détaillées sur le dépôt origin (ou : git remote -v)
git push -u origin master
```

7. Effacer l'historique depuis le premier commit et supprimer les modifications (1 pt)

Solution : revenir au premier commit avec un **reset dur**, puis forcer la mise à jour du dépôt distant.

```
git log --oneline                    # repérer l'identifiant du premier commit, ex. a1b2c3d
git reset --hard a1b2c3d             # supprime les commits suivants ET les modifications des fichiers
git push --force origin master       # le distant avait les anciens commits : il faut forcer
```

Variante sans copier l'identifiant : `git reset --hard $(git rev-list --max-parents=0 HEAD)`.

⚠️ `--hard` efface définitivement les modifications non sauvegardées, et `--force` réécrit l'historique partagé : à utiliser seulement si personne d'autre ne travaille sur la branche. Pour annuler sans réécrire l'historique, on utiliserait plutôt `git revert`.

(Cette question demande seulement de proposer une solution. Elle n'a pas été exécutée sur ce dépôt, pour garder l'historique des autres questions.)

Partie 3 : exercice Git (10 points, 2 points par question)

1. Quelle commande permet de voir l'historique des commits ?

```
git log
git log --oneline     # version courte, une ligne par commit
```

2. Comment créer une branche `feature-xyz` à partir de `main` ?

```
git checkout main
git checkout -b feature-xyz
# ou en une seule commande : git checkout -b feature-xyz main
```

3. Quelle commande fusionne `feature-xyz` dans `main` ?

On se place d'abord sur la branche qui reçoit la fusion :

```
git checkout main
git merge feature-xyz
```

4. Annuler ses modifications locales sur un fichier avant de commiter ?

```
git restore nom_du_fichier
# ancienne syntaxe : git checkout -- nom_du_fichier
```

Si le fichier a déjà été ajouté au staging : `git restore --staged nom_du_fichier` puis `git restore nom_du_fichier`.

5. Comment vérifier l'état actuel du dépôt (fichiers modifiés, ajoutés, en attente) ?

```
git status
```

> Dans ce dépôt, la branche principale s'appelle `master`. Les questions 2 et 3 ont donc été pratiquées avec `master` (`git checkout -b feature-xyz master`, puis `git merge feature-xyz`).